



Universidad Peruana de Ciencias Aplicadas



PixelLang

Un lenguaje para describir la máquina de estados de los personajes de un videojuego 2D

Trabajo Parcial | Teoría de Compiladores

Sección 13962 | Grupo 5 | Docente: Marco Antonio Sobrevilla Cabezudo

U20221G222	Diego Gilmer Ruiz Soto
U202417368	David Miguel Ramos Parihuaman
U20241D937	Antony Rodrigo Quito Ancasi
U202412516	Mateo Alonso Monge Jiménez

Octubre de 2026



La máquina de estados existe, pero no se escribe

Así suele quedar en el código del juego

```
void actualizar() {  
    if (boton_saltar && !saltando) {  
        saltando = true;  
        vy = 2.5;  
    }  
    if (boton_atacar && !saltando) {  
        atacando = true;  
    }  
    if (recibio_golpe) {  
        vida -= 25;  
        aturdido = true;  
    }  
}
```

atacando y **aturdido** pueden quedar en true a la vez: nada lo impide.



Se piensa como una máquina

Reposo, saltando, atacando o aturdido: el personaje cambia de estado con cada evento (Nystrom, 2014).



Se escribe como banderas

Condicionales que revisan banderas sueltas en el bucle del juego (Nystrom, 2014).



Los editores no la ordenan

En Unity los estados son nodos, pero las condiciones de cada transición quedan en paneles separados (Unity Technologies, s. f.).

Un error en la máquina llega al jugador



30

combinaciones de estado y evento que se prueban a mano en un personaje con 5 estados y 6 eventos, cada vez que cambia el diseño

- El compilador del juego no detecta estos errores: para él, las banderas son solo variables.
- Hoy se encuentran jugando. Las pruebas son casi solo partidas manuales (Politowski et al., 2021), y los equipos evitan automatizarlas porque quedan obsoletas cuando cambia el diseño (Murphy-Hill et al., 2014).

PixelLang: cada personaje es un autómata finito

Lo que existe hoy para escribir máquinas de estados como texto

SCXML

Formato XML de propósito general, para intérpretes (World Wide Web Consortium, 2015)

XState

Librería de propósito general para aplicaciones web (Stately, s. f.)

PixelLang

Lenguaje de dominio específico: el autómata de cada personaje se verifica al compilar

Autómata	En PixelLang	Ejemplo
Q	estado	Reposo, Saltando, Aturdido
Σ	eventos	saltar, atacar, recibir_golpe
δ	cuando e -> q	$\delta(\text{Reposo}, \text{saltar}) = \text{Saltando}$
q_0	inicial	Reposo
F	final	Derrotado

$$A = \{Q, \Sigma, \delta, q_0, F\}$$

Un lenguaje de dominio específico permite analizar y verificar los programas en términos del dominio (Mernik et al., 2005).

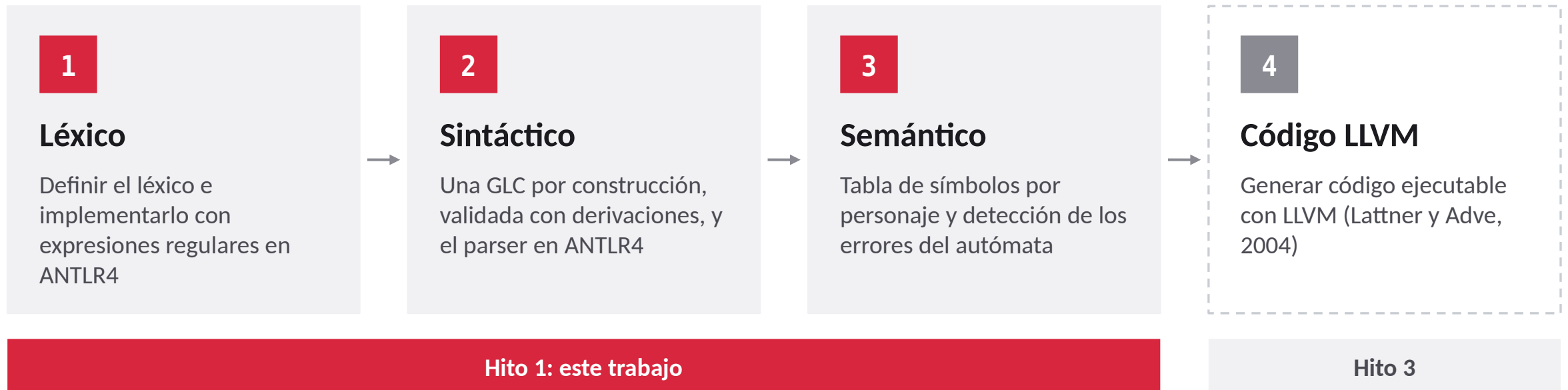
Verificar al personaje es verificar que su autómata esté bien formado.

Objetivo: verificar la máquina al compilar

Objetivo general

Desarrollar un compilador para PixelLang que verifique, al compilar, que la máquina de estados de cada personaje esté bien formada, y que la traduzca a código ejecutable.

Objetivos específicos



Programa, atributos y eventos

1

Programa y personaje ejemplo 1.2

Lista de personajes seguida de simulaciones. Un personaje contiene sus atributos, sus eventos y al menos un estado.

```
personaje Puerta {  
  eventos ser_abierta;  
  inicial estado Cerrada {  
    cuando ser_abierta -> Abierta;  
  }  
  final estado Abierta { }  
}  
simular Puerta: ser_abierta;
```

2

Declaración de atributos ejemplo 2.1

Variable del personaje con nombre, tipo y valor inicial.

```
vida: entero = 100;
```

3

Declaración de eventos ejemplo 3.2

Lista de los eventos del personaje, que forman su alfabeto.

```
eventos saltar, atacar;
```

- Un programa declara uno o más personajes y, al final, cero o más simulaciones. Dentro de un personaje van los atributos, los eventos y los estados, en ese orden.

Estados, transiciones, acciones y expresiones

4

Declaración de estados ejemplo 4.5

Estado con un modificador opcional (inicial o final), acciones al entrar y transiciones.

```
estado Saltando {  
  al_entrar { y = y + 2.5; }  
  cuando aterrizar -> Reposo;  
  cuando recibir_golpe -> Aturdido;  
}
```

5

Transiciones ejemplo 5.2

Evento que la dispara, guarda opcional con si y estado destino.

```
cuando fin_aturdimiento si vida <= 0 -> Derrotado;
```

6

Acciones ejemplo 6.3

Asignación a un atributo y selección si con sino opcional.

```
si vida < 50 { velocidad = 1.0; }
```

7

Expresiones ejemplo 7.4

Operaciones lógicas, relacionales y aritméticas con precedencia.

```
municion > 0 && escudo != verdadero
```

35

ejemplos en total, cinco por construcción (Apéndice A del informe). El parser los acepta todos.

43 tokens definidos con expresiones regulares

10 palabras reservadas

5 tipos y valores lógicos

15 operadores

7 signos de puntuación

3 números e identificadores

3 se descartan: comentarios y espacios

Extracto en la forma token: lista de lexemas

```
CUANDO: cuando      SI: si      FLECHA: ->
LOGICO: logico, lógico  MENOR_IGUAL: <=
NUM_REAL: [0-9]+ '.' [0-9]+
ID: LETRA (LETRA | DIGITO)*
```

Entrada y tokens

```
daño_golpe: entero = 25;
cuando fin_aturdimiento si vida <= 0 -> Derrotado; // comentario
↓
ID DOS_PUNTOS ENTERO ASIGNA NUM_ENTERO PUNTO_COMA
CUANDO ID SI ID MENOR_IGUAL NUM_ENTERO FLECHA ID PUNTO_COMA
```

1

A igual longitud, gana la primera regla

Por eso las palabras reservadas van antes que ID.

2

Siempre gana el lexema más largo

siguiente es un ID y <= es un solo token.

Implementado en ANTLR4 (Parr, 2013).

Una gramática libre de contexto por construcción

G5. Transiciones

transicion → **cuando** *id* *guarda* -> *id* ;

guarda → **si** *expr* | ϵ

G3. Eventos

decl_evs → **eventos** *lista_ids* ;

lista_ids → *lista_ids* , **id** | **id**

- Notación de clase (Aho et al., 2006): no terminales en *cursiva*, terminales en **negrita**, | separa alternativas y ϵ es la cadena vacía.
- **id**, **num** y **dec** representan los tokens ID, NUM_ENTERO y NUM_REAL.
- Las listas usan recursión por la izquierda.

G7. Expresiones

expr → *expr* || *conj* | *conj*

conj → *conj* && *rel* | *rel*

rel → *arit* *oprel* *arit* | *arit*

oprel → < | > | <= | >= | == | !=

arit → *arit* + *term* | *arit* - *term* | *term*

term → *term* * *unario* | *term* / *unario* | *unario*

unario → ! *unario* | - *unario* | *factor*

factor → (*expr*) | **num** | **dec** | **verdadero** | **falso** | **id**

Un no terminal por nivel de precedencia, de menor a mayor: la gramática no es ambigua.

Derivación más a la izquierda del ejemplo 5.2

```
cuando fin_aturdimiento si vida <= 0 -> Derrotado;
```

Paso	Forma sentencial	Producción aplicada
0	<i>transicion</i>	<i>transicion</i> → cuando id guarda -> id ;
1	cuando id guarda -> id ;	<i>guarda</i> → si expr
2	cuando id si expr -> id ;	<i>expr</i> ⇒* id <= num (G7)
3	cuando id si id <= num -> id ;	<i>fin</i>

28

derivaciones más a la izquierda, cuatro por gramática (Apéndice B del informe)

⇒*

resume en un paso la subderivación que pertenece a otra gramática: aquí, expr en G7

Un visitor construye la tabla de símbolos

Programa

```

1  personaje Caballero {
2      vida: entero = 100;
3      eventos saltar, atacar, aterrizar;
4      inicial estado Reposo {
5          cuando saltar -> Saltando;
6          cuando saltar -> Reposo;
7      }
8      estado Saltando {
9          cuando aterrizar -> Reposo;
10         cuando atacar -> Volando;
11     }
12 }

```

Ámbito de Caballero (salida de --tabla)

Nombre	Categoría	Tipo o modificador	Línea
vida	atributo	entero	2
saltar	evento		3
atacar	evento		3
aterrizar	evento		3
Reposo	estado	inicial	4
Saltando	estado		8

Lo que la tabla dice del autómata

$\Sigma = \{\text{saltar, atacar, aterrizar}\}$

$Q = \{\text{Reposo, Saltando}\}$

$q_0 = \text{Reposo}$

$F = \emptyset$

Dos errores del autómata: E01 y E03

E01 Transición hacia un estado no declarado

Regla: δ solo puede llevar a estados de Q

```
8      estado Saltando {  
9          cuando aterrizar -> Reposo;  
10         cuando atacar -> Volando;  
11     }
```

Error semántico E01 en la línea 10, columna 25:
la transición de 'Saltando' con 'atacar' va hacia 'Volando', que no es un estado declarado

E03 Transiciones no deterministas

Regla: $\delta(q, e)$ tiene un único destino

```
4      inicial estado Reposo {  
5          cuando saltar -> Saltando;  
6          cuando saltar -> Reposo;  
7     }
```

Error semántico E03 en la línea 6, columna 8:
en 'Reposo', la transición con 'saltar' de la línea 6 nunca se toma: la de la línea 5 responde al mismo evento sin guarda

■ El compilador verifica otros ocho errores semánticos (Apéndice C del informe); cada uno tiene su programa de prueba.

Lexer y parser en ANTLR4, probados con varias entradas

Repositorio github.com/DavidRamPari/pixellang

```
gramatica/PixelLang.g4  léxico y sintaxis
src/main.py             driver
src/frontend.py         lexer y parser de ANTLR4
src/semantico.py        visitor: tabla de símbolos, E01 a E10
ejemplos/               caballero.pxl, slime.pxl
pruebas/               léxicas, sintácticas y semánticas
docs/                  informe y presentación
```

Comandos

```
python src/main.py ejemplos/caballero.pxl
    --tokens --arbol --tabla
python pruebas/correr_pruebas.py
python pruebas/probar_construcciones.py
```

■ Gestión en GitHub: un hito por entrega e issues por tarea.

Pruebas de los analizadores (Apéndice D)

6

Léxico

1 entrada válida y 5 con caracteres o lexemas inválidos

41

Sintáctico

35 ejemplos aceptados y 6 programas con errores rechazados

10

Semántico

Un programa por cada error, de E01 a E10

Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.ª ed.). Pearson.

Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3), 231–274. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)

Lattner, C. y Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis & transformation. En *International Symposium on Code Generation and Optimization, 2004. CGO 2004* (pp. 75–86). IEEE. <https://doi.org/10.1109/CGO.2004.1281665>

Mernik, M., Heering, J. y Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316–344. <https://doi.org/10.1145/1118890.1118892>

Murphy-Hill, E., Zimmermann, T. y Nagappan, N. (2014). Cowboys, ankle sprains, and keepers of quality: How is video game development different from software development? En *Proceedings of the 36th International Conference on Software Engineering* (pp. 1–11). ACM. <https://doi.org/10.1145/2568225.2568226>

Nystrom, R. (2014). State. En *Game programming patterns*. Genever Benning. <https://gameprogrammingpatterns.com/state.html>

Parr, T. (2013). *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf.

Politowski, C., Petrillo, F. y Guéhéneuc, Y.-G. (2021). *A survey of video game testing* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2103.06431>

Stately. (s. f.). *XState*. Recuperado el 4 de octubre de 2026, de <https://stately.ai/docs/xstate>

Unity Technologies. (s. f.). *Introduction to Animator Controllers*. Recuperado el 4 de octubre de 2026, de <https://docs.unity3d.com/Manual/class-AnimatorController.html>

World Wide Web Consortium. (2015). *State chart XML (SCXML): State machine notation for control abstraction* (W3C Recommendation). <https://www.w3.org/TR/scxml/>

El compilador en funcionamiento



Video de demostración: <https://drive.google.com/file/d/1asrzQjXH87vR6JPZDanqDFVFOEntwdZ8/view?usp=sharing>